Dynamic DB Copilot

Comprehensive Architecture & LLM Engineering Guide

What is this project?
Dynamic DB Copilot is a multi-service developer utility that translates natural language questions into database queries (PostgreSQL SQL or MongoDB MQL). It validates queries for security, executes them, and automatically corrects errors using an AI Self-Healing loop powered by the Google Gemini API.

1. Three-Tier Architecture

The project is structured into three isolated, communicating layers to maintain modularity and safety:

- Frontend Client (React/Vite): Built in Javascript. It connects databases (or mock databases), visualizes the live schemas, holds chat conversation history, displays natural language explanations, and shows tables of queried data.

- API Gateway (Express.js): Built in Node.js. It acts as an entry point. It enforces API key authorization (x-api-key) and rate-limits requests to ensure the backend is secure.

- AI Engine (FastAPI): Built in Python. It executes introspection scripts on target databases, sends prompt contexts to Gemini, performs safety AST checks, runs the queries, and implements the dynamic self-healing retries.

2. Technology Stack & AI Integration Choices

Why Direct Google GenAI SDK (No LangChain)?

Unlike many AI apps, this project does NOT use LangChain or LangGraph. Here is why:

1. Reduced Overhead: LangChain adds multiple layers of abstraction that can slow down API requests.

2. Fine-grained Prompt Control: By formatting the prompts directly in Python, we retain absolute control over the system instructions and formatting rules.

3. Pydantic Compatibility: Google GenAI SDK provides direct JSON-schema formatting. Since we bypass framework parsers, we avoid validation bugs and get reliable, structured JSON outputs directly from the model.

Is it RAG (Retrieval-Augmented Generation)?

Yes, but it is a Structured Schema Retrieval rather than Vector-based RAG. Here is how it works:

- Standard RAG converts document chunks into vectors and searches a vector database (like Chroma or Pinecone).
- This project queries database metadata (the system catalogs in PostgreSQL or sample documents in MongoDB).
- It retrieves the current structure of the database (tables, columns, types, nested fields, sample data) and injects this layout directly into the prompt context.
- This allows the LLM to understand the database structure accurately without needing a separate vector index.

3. Prompt Engineering & LLM Techniques

System Instructions (Developer Prompts)

We configure the LLM behavior using permanent System Instructions. These act as strict rules that the model cannot ignore:
- Role definition: 'You are a senior PostgreSQL DBA / MongoDB Developer'.
- Safety constraints: 'Generate ONLY read-only queries (SELECT/WITH for SQL or find/aggregate for Mongo)'.
- Off-Topic Guardrail: 'If the user question is unrelated to the database schema, you MUST return the message "I am specialized to answer database-related questions only."'

Few-Shot Prompting (Learning from Examples)

To teach the LLM how to parse complex database queries, we inject few-shot examples directly into the prompt context. For MongoDB, we supply exact user-question-to-JSON-payload mapping examples, illustrating how to filter string fields using case-insensitive regular expressions ('{$regex: ..., $options: "i"}') and run aggregation pipelines. This guarantees formatting accuracy.

Iterative Error-Feedback (Self-Healing)

If a generated query is executed but fails (e.g., due to a column name mistake or a bad cast), the self-healer captures the exact database traceback error. It appends the failed query and the traceback detail to a new prompt context and asks Gemini to fix it. The model uses this feedback loop (up to 3 attempts) to dynamically correct syntax and execute a working query.

4. Core Component Code Walkthrough

A. Introspective Schema Crawler (app/introspector.py)

Reads the database structure directly:
- Postgres: Queries catalog views to inspect schema, tables, column names, column data types, nullable flags, primary keys, and foreign keys.
- MongoDB: Connects to the database collections, queries a sample of documents to dynamically map nested JSON fields/keys, detects data types, and records sample lists to help the LLM identify filters.

B. Safety Guardrail Validator (app/security.py)

Analyzes query commands before they reach the database:
- Postgres: Scans the SQL command string and ensures it matches read-only patterns (SELECT or WITH). Blocks mutations (INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, etc.) and raises permission errors.
- MongoDB: Validates the target operation against a list of safe read-only operations (find, find_one, count_documents, distinct, aggregate, db_stats). It blocks write operations (insert, update, delete, etc.).

C. Query Orchestrator & Self-Healer (app/self_healer.py)

Maintains the lifecycle of a request:
1. Builds prompts by pulling schema metadata and few-shots.
2. Directs Gemini to generate a structured JSON query payload (using response_mime_type: 'application/json').
3. Runs security validation checks.
4. Attempts database execution. If database throws an exception, feeds the error back to Gemini for correction.
5. Passes execution results to a final Gemini summarizer to compile a friendly, developer-oriented natural reading answer.